

Детекция на Point Cloud
GitHub: [Point-Cloud-Detection](#)
(Отчет по проекту №4)

Выполнили студентки:

группы БПМИ2310, 3 курса
группы БПМИ233, 3 курса

Киселева Анна Юрьевна
Трунина Александра Алексеевна

Руководитель проекта:

Алишер Ибрагимов

Содержание

1	Анализ текущих SOTA	3
1.1	PointNet [2]	3
1.2	PointNet++ [3]	4
1.3	PointTransformer [4, 5, 6]	4
1.4	Sonata [7]	5
1.5	PointPillars [8]	5
2	Датасеты	7
3	Метрики	8
4	Baseline for outdoor	9
4.1	Дообученная модель	9
4.1.1	Данные	9
4.1.2	Архитектура модели	9
4.1.3	Дообучение	10
4.1.4	Результаты	10
4.2	Baseline с готовыми весами модели	10
5	Работа с LiDAR	12
6	Запуск модели	13
7	План дальнейшей работы	15
7.1	Демонстрационное приложение	15
7.2	Собственная тестовая выборка	15

1 Анализ текущих SOTA

При работе с Point Cloud есть следующие особенности:

- Неупорядоченность точек: перестановка точек (порядок, в котором они подаются на вход сети) в облаке не должна менять результат
- Наличие структуры: близлежащие точки, как правило, образуют какую-то локальную структуру и свойства между ними вероятно сохраняются, либо незначительно изменяются (например цвет). Модель должна этому соответствовать.
- Устойчивость к аффинным преобразованиям: объект не превращается в другой объект при применении линейного отображения и сдвигу к нему. Модель должна быть устойчивой к этому (аналогично как CNN устойчивы к сдвигам объекта на 2d картинке)

Раньше люди дискретизовали точки и работали с ними как с вокселями, используя стандартные методы в духе CNN. Однако у этого подхода множество минусов. Одни из ключевых – это необходимость в высоком качестве дискретизации, из чего следуют проблемы с вычислительными ресурсами. Также данные в 3D облаке зачастую очень разрежены, что также влечет ухудшение качества и увеличивает время сходимости. Поэтому было необходимо придумать принципиально новый метод.

1.1 PointNet [2]

Первое продвижение случилось с выходом статьи PointNet. Авторы отошли от старых подходов и сделали акцент на том, чтобы модель архитектурно учитывала инвариантность относительно изменения порядка точек.

Дальше пусть мы решаем задачу классификации облака (задача семантической сегментации аналогична). На вход нам подается батч размерности $X \in \mathbb{R}^{B \times N \times 3}$, B - размер батча, N - максимальное количество точек в объекте (если изначально больше, то сэмплим N). Пайплайн решения такой:

- 1 В начале мы применяем T-Net - сеть для выравнивания точек. Это нужно для инвариантности относительно аффинных/геометрических преобразований. T-net это 3-слойная MLP, которая переводит наши точки в пространство $\mathbb{R}^{1024}$, затем делается max pooling вдоль оси точек (то есть был тензор $(B, N, 1024)$, стал $(B, 1024)$), и затем еще одна 3-слойная MLP переводящая в $\mathbb{R}^9$. Затем мы это решаем в матрицу 3×3 , прибавляем I и применяем эту матрицу к изначальному x . Прибавление единичной матрицы нужно, чтобы по умолчанию нейросеть не изменяла данные, а затем в процессе при необходимости выучивала их нормализацию. Здесь ключевое это применение max pooling и обработка точек MLP независимо – это и дает модели инвариантность относительно порядка
- 2 Дальнейшие шаги во многом аналогичны первому в плане архитектуры, поэтому распишу не так подробно. Второй шаг это применить неглубокую MLP к нормализованным данным, получив достаточно сырые фичи исходных точек
- 3 Тут мы применяем трюк, аналогичный первому – только теперь мы нормализуем не данные, а фичи от них. Архитектура практически та же, но в силу более высокой размерности модели будет сложно выучить качественную матрицу преобразования. Поэтому дополнительно применяется регуляризация, которая форсирует модель выучивать ортогональную матрицу – $\|I - T^T T\|_F^2$, где T это матрица, которую предсказала модель
- 4 Наконец применяем глубокую MLP, затем max pooling и финальную MLP. В финальной MLP последний линейный слой возвращает количество логитов, соответствующее количеству классов
- 5 Во всех MLP также применяется одномерный BatchNorm, ReLU. Dropout применяется только в финальной MLP

1.2 PointNet++ [3]

Проблема PointNet она плохо выучивает локальные признаки данных, что негативно сказывается на качестве. PointNet++ исправляет эту проблему.

Посмотрим на CNN - в начале они изучают локальные признаки, за счет маленького receptive field (в начале оно маленькое за счет маленького kernel size). Затем, чем глубже CNN, тем больше становится receptive field. Таким образом с ростом глубины, свертки начинают учить более глобальные признаки, чем в начале, при этом не забывая про локальные, выученные в начале. Это позволяет им комбинировать знания и о локальных, и о глобальных зависимостях.

Авторы сделали аналогичное для PointNet. Вместо независимой обработки точек, давайте обработаем малые их окрестности, затем комбинируя их будем постепенно выучивать глобальные признаки, имитируя таким образом поведение CNN.

Более конкретно:

- 1 Sampling Layer. На этом этапе мы сэмплим точки, которые в дальнейшем будут выступать центрами наших окрестностей, то есть центроидами. При этом мы хотим, чтобы точки как можно более равномерно покрывали объект, поэтому наивное сэмплирование не подходит. Авторы используют FPS алгоритм – первую точку сэмплим равномерно, на каждой следующей итерации мы выбираем самую дальнюю точку от текущего множества выбранных. Это позволяет лучше покрыть облако
- 2 Grouping Layer. На этом этапе мы хотим найти окрестности. Тут два подхода – либо мы берем для каждой точки всех соседей в заранее фиксированном радиусе (и также ограничив их верхней границей K), либо применяем kNN, чтобы найти ближайшие K соседей. Первый подход гарантирует фиксированный размер окрестности, что позволяет модели лучше обобщать локальные признаки
- 3 PointNet Layer. В каждой окрестности мы нормализуем точки относительно центроида. Затем применяем mini версию PointNet для каждой окрестности, получая соответствующий ей вектор признаков. Таким образом операции агрегации (нормализация, max pooling) будут применяться внутри окрестности, извлекая именно локальные признаки
- 4 После применения вышеуказанных слоев, мы получим feature вектора каждой окрестности. Далее мы применяем аналогичные слои Sampling -> Grouping -> PointNet, но теперь на вход идут не изначальные точки, а полученные с прошлого слоя вектора окрестностей. При этом на каждом слое мы уменьшаем число центроидов. Таким образом мы постепенно увеличиваем receptive field и переходим от локальных, к более глобальным признакам, что и требовалось

1.3 PointTransformer [4, 5, 6]

В 2017 с выходом статьи "Attention is all you need" трансформеры стали использовать везде где только можно. Отсюда появилась статья PointTransformer, которая де факто взяла за основу PointNet++ и заменила в нем MLP механизм на трансформер.

Слои Sampling и Grouping оставим теми же. Далее применим стандартный трансформер со следующими изменениями:

- Self attention применяется не глобально, а внутри окрестности, которые мы получили с прошлого этапа
- В отличие от стандартного, тут self attention векторный, то есть мы получаем разные веса для разных каналов
- Сам attention выглядит немного по-другому – вместо умножения ключей на запросы, мы вычитаем их. Также positional encoding мы прибавляем не только к ключам/запросам, но и к значениям

- Вместо стандартного position encoding используется relative position encoding. Также вместо наивно обучаемых эмбеддингов, используется MLP. Это позволяет лучше кодировать геометрию

1.4 Sonata [7]

Предыдущие модели явно решали задачу классификации/сегментации. Важным классом моделей являются SSL модели. Одним из ярких их представителей в случае 2d CV является DINOv2. Авторы хотели создать аналогичное для обработки облака точек, ведь попытки до них набирали достаточно посредственное качество.

Одним из ключей к обучению хорошей SSL модели является постепенное усложнение задачи. Модель за счет сложности будет выучивать более верхнеуровневую семантику объектов, что позволит выучить высокое качество эмбеддингов. В случае картинок проблемы нет – если, например, замаскировать (закрасить черным) участок фотографии, то модель потеряет информацию о нем и ей придется восстанавливать информацию из соседних участков. В случае облака точек все сложнее – координаты точки уже содержат геометрическую информацию о ней и наивное маскирование не работает. Вместо выучивания высокоуровневой семантики, модель "читерит" и выучивает геометрию, что пагубно влияет на качество.

Чтобы исправить этот недостаток, авторы предлагают взять стандартный пайплайн обучения DINOv2, в качестве backbone взять PointTransformer и добавить следующие изменения:

- Убрать полностью декодер из трансформера, обучать только энкодер. Интуиция следующая: энкодер постепенно понижает размерность картинок, делая признаки все более "размытыми" а доступ к геометрической информации все более сложным. Декодер же наоборот повышает размерность и форсирует модель выучивать геометрию. Вместо этого мы будем декодировать без обучаемой модели. А именно, мы будем запоминать в какие окрестности мы группировали на каждом слое и поэтапно склеивать признаки точки с признаками её окрестности. Таким образом мы сильно уменьшаем утечку геометрической информации в модель
- Для маскирования точек авторы используют добавление гауссовского шума к координатам с большой дисперсией. Это не позволит модели сильно полагаться на геометрическую информацию
- Маскирование делается постепенно, его доля увеличивается от 30% до 70% в течение первых 5% эпох обучения. Интуиция в том, что если сразу сделать сильно маскирование, то модель выберет самый простой путь решения - геометрический. Если же планомерно повышать, то у модели будет шанс выучить семантические признаки и не переключаться на геометрические в процессе усложнения задачи

1.5 PointPillars [8]

PointPillars — архитектура для детекции объектов в облаках точек, ориентированная на высокую скорость инференса при конкурентном качестве. Ключевая идея представить облако точек в виде вертикальных столбцов (pillars) и свести задачу к стандартному 2D детектированию на псевдо-изображении.

На вход подаётся облако точек $X \in \mathbb{R}^{N \times 4}$, где каждая точка описывается координатами (x, y, z) и интенсивностью сигнала. Пайплайн устроен следующим образом:

- 1 **Pillarization.** Плоскость XY разбивается на равномерную сетку, каждая ячейка которой образует вертикальный столбец pillar. Для каждой точки внутри столбца вычисляется расширенный вектор признаков, включающий исходные координаты, интенсивность и смещения относительно центра pillar.

- 2 **Pillar Encoder.** Каждый pillar обрабатывается независимо: линейный слой с BatchNorm и ReLU, затем max pooling по всем точкам столбца. В результате каждый pillar описывается одним вектором признаков по аналогии с тем, как PointNet извлекает глобальный дескриптор из набора точек.
- 3 **Scatter.** Векторы признаков pillars размещаются обратно на сетку XY , формируя 2D псевдо-изображение в проекции сверху BEV. После этого шага облако точек превращается в обычный тензор изображения.
- 4 **2D Backbone и Neck.** Псевдо-изображение обрабатывается стандартной свёрточной сетью с несколькими уровнями понижения разрешения. Признаки с разных масштабов объединяются через FPN-подобную структуру, обеспечивая мультимасштабное представление сцены.
- 5 **Detection Head.** На основе заранее заданных анкеров голова предсказывает для каждой позиции сетки логиты классов, регрессионные поправки к координатам и размерам бокса, а также угол поворота. На инференсе применяются пороговая фильтрация и NMS.

Функция потерь включает три компоненты:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{dir}} \quad (1)$$

где $\mathcal{L}_{\text{cls}}$ — focal loss для классификации, $\mathcal{L}_{\text{reg}}$ — SmoothL1-лосс для регрессии координат и размеров бокса, $\mathcal{L}_{\text{dir}}$ — cross-entropy лосс для классификации направления.

Главное преимущество PointPillars перед предшественниками — отсутствие затратных 3D свёрток: вся обработка сводится к быстрой операции pillarization и стандартным 2D свёрткам.

2 Датасеты

- **SUN RGB-D** [10] является одним из наиболее подходящих датасетов для indoor-сцен. Он содержит 10 тысяч кадров, полученных с разных RGB-D сенсоров. Каждая сцена размечена в 3D bounding boxes, охватывающими множество категорий объектов интерьера: стулья, столы, кровати, полки и другие предметы. Наличие цвета для каждой точки может быть полезно при детекции, так как цвет по сути дополнительный признак, помогающий различать объекты.
- **ScanNet** [11] представляет собой датасет из более чем 1500 3D-реконструкций реальных помещений, используется для семантической сегментации в Point Transformer V2 [5] и V3 [6], а также в нём есть 3D bounding boxes, что позволяет использовать его для обучения детекции. В ScanNet точки распределены неравномерно, то есть вблизи сенсора плотность выше, а на удалении точки разрежены, что соответствует реальным сценариям съёмки.
- **S3DIS** [12] включает размеченные по 12 семантическим классам: структурные элементы зданий, мебель и отдельный класс clutter для остального. В нём содержатся координаты (x, y, z) для каждой точки, цвета RGB, нормали поверхности. Этот датасет применяется в PointNet [2], PointNet++ [3], а также во всех версиях Point Transformer [4, 5, 6] для семантической сегментации крупных сцен. Разметка по классам может быть переработана для задачи детекции.
- **KITTI** [13] предназначен для задач на открытых пространствах и автономного вождения. Датасет содержит данные с LiDAR, которые возвращают облака точек с информацией об интенсивности сигнала. Разметка представлена в виде 3D bounding boxes для автомобилей, пешеходов и велосипедистов. KITTI позволяет проверить модель в условиях сильной разреженности данных на дальних расстояниях.
- **ShapeNetPart** [14] используется в PointNet [2], PointNet++ [3] и Point Transformer [4] для задачи сегментации частей объектов. Он содержит 16 881 3D моделей из 16 классов (столы, стулья, самолеты, машины и др.); каждая модель размечена по частям (например, у стула - ножки, сиденье, спинка), что позволяет обучать модели различать не только целые объекты, но и их составляющие.
- **nuScenes** [15] упоминается в Point Transformer V3 [6] как один из ключевых датасетов для outdoor-сцен при мультидатасетном обучении. Он содержит данные с LiDAR-сенсоров, радаров и камер, собранные в Бостоне и Сингапуре. Датасет включает 1000 сцен длительностью 20 секунд, с полной разметкой 3D bounding boxes для 23 классов, включая автомобили, пешеходов, велосипедистов, а также различные элементы инфраструктуры.
- **Warehouse LiDAR** [16] — открытый датасет складских интерьеров, опубликованный на Hugging Face. Содержит облака точек, полученные с промышленного LiDAR-сенсора в условиях реального склада. Каждая сцена размечена 3D bounding boxes для объектов класса Box (картонные коробки). Данные хранятся в формате .pcd и содержат координаты (x, y, z) и интенсивность сигнала. Датасет использован в данной работе в качестве indoor-бейзлайна как пример промышленного применения LiDAR-детекции.

3 Метрики

Для оценки качества детекции объектов в 3D используется метрика Average Precision, так как она учитывает не только правильность определения класса, но и точность предсказанного положения в пространстве.

Для каждого предсказанного объекта рассчитывается пересечение с истинным объектом: Intersection over Union ($IoU = \frac{A \cap B}{A \cup B}$). Выбор порога зависит от конкретной задачи.

Average Precision для заданного класса при фиксированном пороге IoU представляет собой площадь под кривой Precision-Recall, которая строится по результатам детекции объектов данного класса. Итоговой метрикой, характеризующей модель в целом, является mean Average Precision, то есть среднее значение AP по всем классам.

Дополнительной метрикой, используемой в современных исследованиях для оценки качества предобученных представлений, выступает точность при линейном пробировании [7]. Данный подход широко применяется в задачах self-supervised learning для проверки того, насколько хорошо модель научилась выделять семантически значимые признаки. Предобученная модель фиксируется (ее веса не обновляются), а поверх полученных от нее признаков обучается только линейный классификатор. Низкая точность при линейном пробировании свидетельствует о низком качестве эмбедингов, получаемых из SSL модели. Для преодоления этой проблемы в работе Sonata предлагаются две стратегии: маскирование пространственной информации (obscuring spatial information) путем применения self-supervised learning на более грубых пространственных масштабах и усиление зависимости от входных признаков (emphasizing input features). Это позволяет модели избежать сжатия представлений к примитивным геометрическим свойствам и научиться выделять семантически значимые характеристики.

4 Baseline for outdoor

4.1 Дообученная модель

4.1.1 Данные

В качестве датасета использованы данные складов Warehouse LiDAR [16], опубликованные на платформе Hugging Face. Для упрощённого теста была выбрана детекция одного класса Box. Данные были разбиты следующим образом: 128 сцен для обучения, 32 для валидации, 16 для теста.

Облака точек содержатся в формате .pcd, затем конвертируются в бинарный формат .bin, а разметка переводится в текстовый формат:

$$\text{class_name } x \ y \ z \ d_x \ d_y \ d_z \ \text{yaw} \quad (2)$$

где class name — название класса объекта, (x, y, z) — центр бокса, (d_x, d_y, d_z) — размеры бокса, yaw — угол поворота вокруг оси Z .

4.1.2 Архитектура модели

Использована реализация PointPillars [8]. Как уже подробно обозревалось выше, это архитектура для детекции объектов в облаках точек, ориентированная на высокую скорость инференса при конкурентном качестве. Ключевая идея заключается в том, что хотим представить облако точек в виде вертикальных столбцов (pillars) и свести задачу к стандартному 2D детектированию на псевдо-изображении.

На вход подаётся облако точек $X \in \mathbb{R}^{N \times 4}$, где каждая точка описывается координатами (x, y, z) и интенсивностью сигнала. Пайплайн устроен следующим образом:

- 1 **Pillarization.** Плоскость XY разбивается на равномерную сетку, каждая ячейка которой образует вертикальный столбец pillar. Для каждой точки внутри столбца вычисляется расширенный вектор признаков, включающий исходные координаты, интенсивность и смещения относительно центра pillar.
- 2 **Pillar Encoder.** Каждый pillar обрабатывается независимо: линейный слой с BatchNorm и ReLU, затем max pooling по всем точкам столбца. В результате каждый pillar описывается одним вектором признаков по аналогии с тем, как PointNet извлекает глобальный дескриптор из набора точек.
- 3 **Scatter.** Векторы признаков pillars размещаются обратно на сетку XY , формируя 2D псевдо-изображение в проекции сверху BEV. После этого шага облако точек превращается в обычный тензор изображения.
- 4 **2D Backbone и Neck.** Псевдо-изображение обрабатывается стандартной свёрточной сетью с несколькими уровнями понижения разрешения. Признаки с разных масштабов объединяются через FPN-подобную структуру, обеспечивая мультимасштабное представление сцены.
- 5 **Detection Head.** На основе заранее заданных анкерных головок предсказывает для каждой позиции сетки логиты классов, регрессионные поправки к координатам и размерам бокса, а также угол поворота. На инференсе применяются пороговая фильтрация и NMS.

Функция потерь включает три компоненты:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{dir}} \quad (3)$$

где $\mathcal{L}_{\text{cls}}$ — focal loss для классификации, $\mathcal{L}_{\text{reg}}$ — SmoothL1-лосс для регрессии координат и размеров бокса, $\mathcal{L}_{\text{dir}}$ — cross-entropy лосс для классификации направления.

Главное преимущество PointPillars перед предшественниками — отсутствие затратных 3D свёрток: вся обработка сводится к быстрой операции pillarization и стандартным 2D свёрткам.

4.1.3 Дообучение

Модель обучалась на GPU на Kaggle, T400, в течение 6 эпох (см. Рисунок 4.1). Функция потерь включает три компоненты:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{dir}} \quad (4)$$

где $\mathcal{L}_{\text{cls}}$ — focal loss для классификации, $\mathcal{L}_{\text{reg}}$ — регрессионный лосс для координат и размеров бокса, $\mathcal{L}_{\text{dir}}$ — лосс классификации направления.

	epoch	train_total_loss	train_cls_loss	train_reg_loss	train_dir_loss	val_avg_num_preds	val_avg_score
0	1	1.721430	0.754989	0.424375	0.588453	0.0000	0.000000
1	2	0.928050	0.351177	0.235329	0.531075	0.0000	0.000000
2	3	0.587894	0.220227	0.141259	0.425741	0.0000	0.000000
3	4	0.389607	0.122655	0.099389	0.340871	0.5625	0.051744
4	5	0.253812	0.071529	0.065411	0.257306	1.8750	0.055420
5	6	0.184893	0.046793	0.050703	0.183468	6.5000	0.057256

Рисунок 4.1: Дообучение модели

4.1.4 Результаты

Качество инференса можно увидеть по BEV-проекции (см. Рисунок 4.2). Детектор демонстрирует работоспособность, однако уверенность предсказаний остаётся низкой, а число ложных срабатываний велико, что объясняется малым объёмом обучающих данных и небольшим числом эпох.

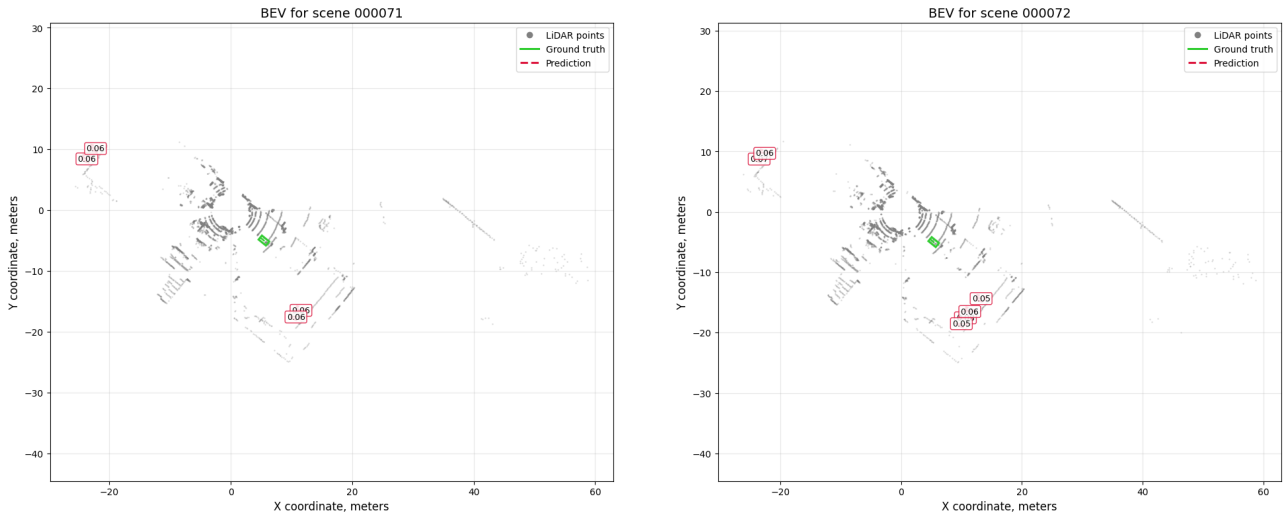


Рисунок 4.2: Результаты детекции объектов на складском датасете (вид сверху)

4.2 Baseline с готовыми весами модели

В данном случае был запущен baseline, где в качестве модели использовалась архитектура PointPillars с предобученными весами. Дообучение модели не выполнялось, использовался готовый checkpoint, обученный ранее на данных KITTI. В ноутбуке решалась задача именно инференса и базовой оценки качества.

- Подготовка. Для совместимости библиотек и минимизации конфликтов в Kaggle использовалась облегчённая реализация PointPillars на PyTorch без тяжёлых 3D зависимостей и

без дополнительной сборки CUDA/C++ расширений. Затем были загружены веса модели и сформированы входные данные LiDAR.

- Инференс. Облако точек подавалось на вход модели, после чего модель возвращала 3D боксы, метки классов объектов и оценки уверенности в них. Полученные предсказания визуализировались на рисунке (см. Рисунок 4.3)

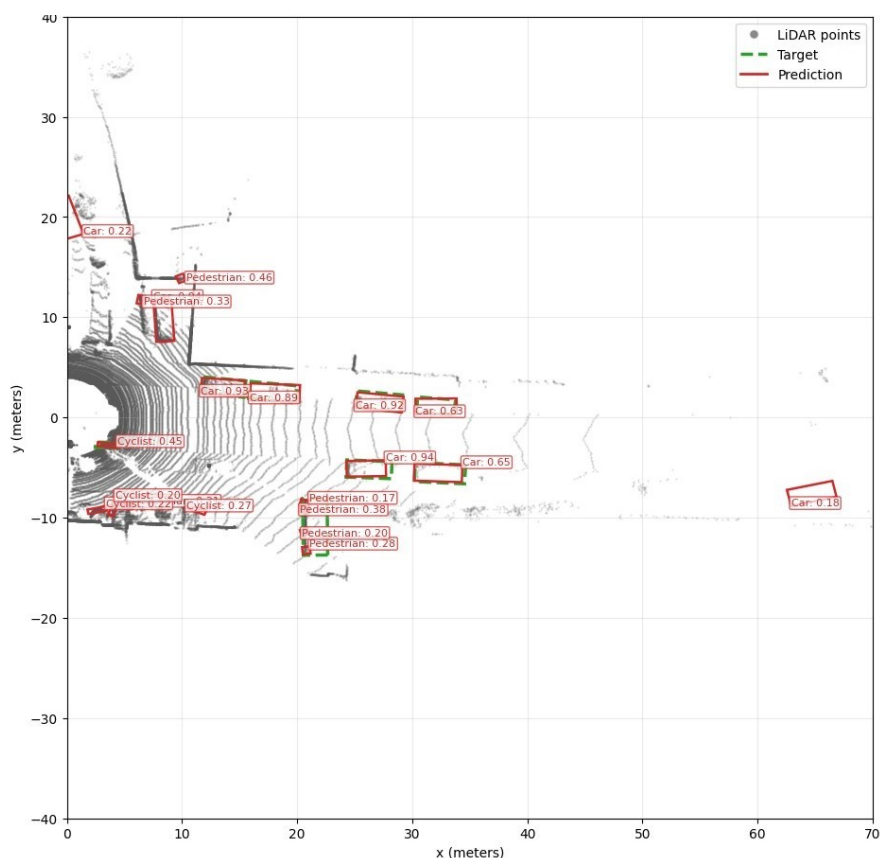


Рисунок 4.3: Результаты детекции объектов предобученной модели

- Оценка качества. После этого была проведена оценка качества на небольшом подмножестве KITTI. Для каждого объекта сравнивались предсказанные и целевые 3D боксы, после чего вычислялась метрика 3D mAP и значения AP по классам. Также были построены precision-recall график и столбчатая диаграмма по классам.
- Итоги и метрики. Итоговый результат показал, что бейзлайн работает корректно: модель принимает LiDAR данные на входе и выдаёт классифицированные 3D боксы на выходе. На eval для класса Car был получен AP около 0.3, для Cyclist около 0.23, итоговый 3D mAP составил примерно 0.176. Для класса Pedestrian значение оказалось нулевым, так как в использованной мини-выборке отсутствовали объекты этого класса в таргетах (см. Рисунок 4.4)

	class	iou_thr	num_gt	num_pred	AP_3D	AP_3D_pct
0	Car	0.700000	17	56	0.3007	30.07%
1	Pedestrian	0.500000	0	59	0.0000	0.00%
2	Cyclist	0.500000	3	34	0.2273	22.73%

Рисунок 4.4: Значения метрик

5 Работа с LiDAR

На текущем этапе отработан полный пайплайн получения и обработки LiDAR-данных с iPad и iPhone. Реализованы два способа получения данных:

Стриминг в реальном времени. С помощью приложения Record3D и скрипта `ipad_stream.py` [17] iPad подключается к компьютеру по USB или сети, после чего облако точек появляется в окне визуализатора в реальном времени. Каждый кадр содержит карту глубины и синхронизированный RGB-кадр, из которых восстанавливается облако точек с цветом.

Экспорт файла .r3d. Запись сохраняется в формате `.r3d` — zip-архив, содержащий покадровые карты глубины (`.depth`), маски уверенности (`.conf`), RGB-кадры (`.jpg`) и файл метаданных с параметрами камеры. Написан код для работы с этим форматом: распаковка `depth` через `lzfse`, восстановление облака точек через обратную проекцию с использованием матрицы интринсиков K , совмещение с RGB для получения цвета, фильтрация по уверенности сенсора.

На выходе для каждого кадра получается облако точек в формате (x, y, z, r, g, b) , пригодное для подачи в модель. Дополнительно реализована первая попытка сборки `fused cloud` — объединение нескольких кадров в одно плотное облако с использованием `pose`-информации из метаданных, но для полноценной работы с ним требуются более сложные подходы.

Таким образом, на данный момент имеется: доступ к глубине, цвету и нормальям поверхности с LiDAR-сенсора, отлаженный код парсинга `.r3d`, интерактивная визуализация облаков точек.

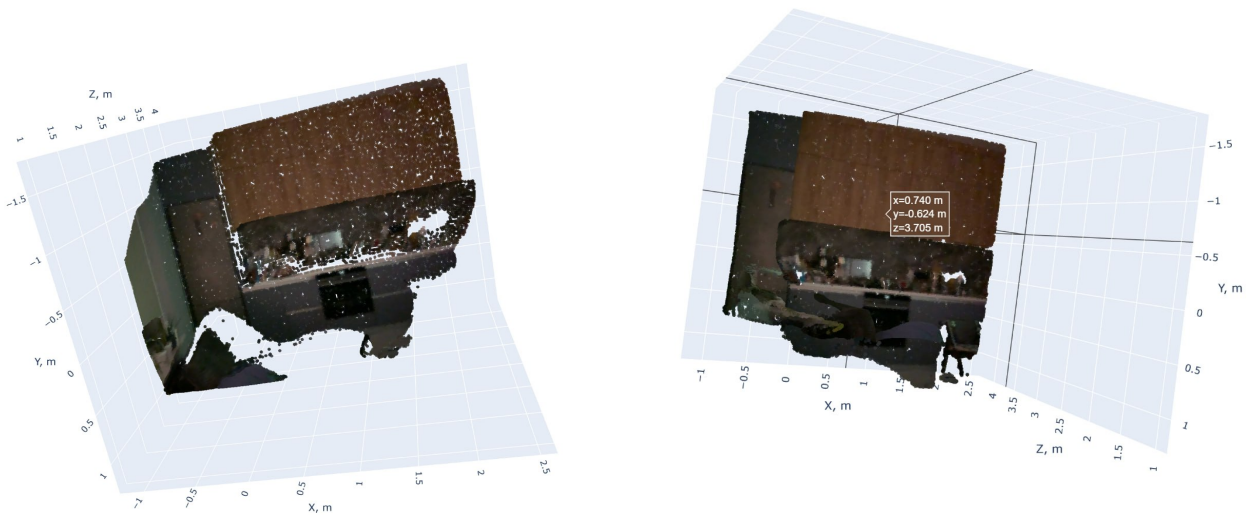


Рисунок 5.1

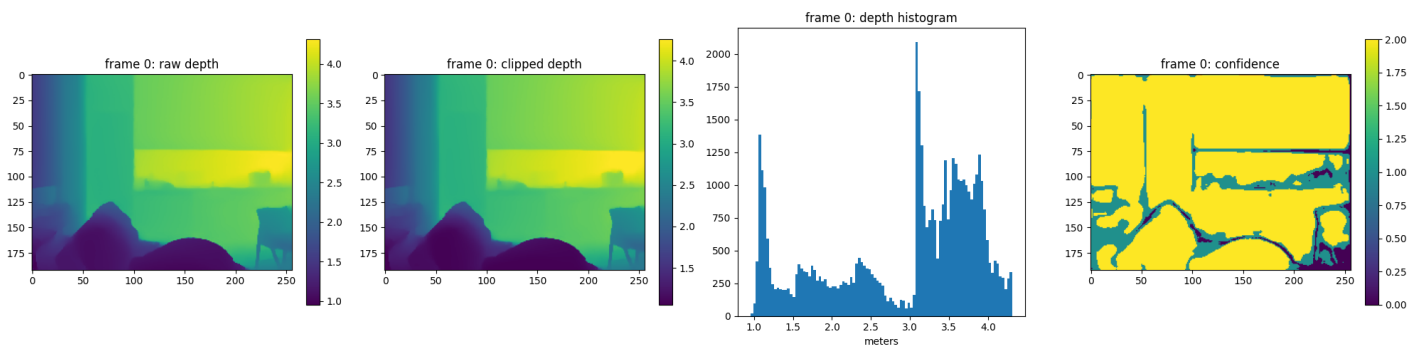


Рисунок 5.2: Карта глубины и гистограмма распределения расстояний

6 Запуск модели

На данном этапе была запущена архитектура Sonata. Как уже написано выше, данная архитектура ориентирована не на прямую детекцию, а на получение надёжных признаков точек без использования полной разметки на этапе предобучения. В качестве реализации использовался официальный репозиторий [facebookresearch/sonata](https://github.com/facebookresearch/sonata). Для запуска были подготовлены совместимые версии библиотек через conda, после чего был загружен предобученный чекпоинт модели.

На вход Sonata подаётся облако точек, а на выходе модель возвращает эмбединги точек, содержащие семантически значимые признаки. В базовом режиме работы это именно признаки, а не 3D bounding boxes. В рамках данной работы Sonata была нужна как indoor-бейзлайн, позволяющий получить качественное представление сцен интерьера и использовать их как промежуточный шаг между облаком точек и задачей выделения отдельных объектов в боксы.

Запуск модели в Kaggle был успешно выполнен. Была собрана отдельная среда, после чего проведён полный проверочный инференс на примере. В результате модель корректно отработала на GPU и вернула признаки исходного облака точек размерности (273530, 1088), что подтверждает работоспособность пайплайна загрузки данных, инференса и обратного отображения признаков к исходному масштабу. Дополнительно были построены диагностические графики, отражающие распределение норм признаков и PCA-проекции, что позволило проверить, что выходные эмбединги не вырождаются и содержат осмысленную структуру.

Также была выполнена визуализация результатов запуска. Поскольку стандартные Open3D-визуализации работают в Kaggle нестабильно из-за headless-режима, демонстрационные скрипты репозитория были переделаны так, чтобы сохранять результаты в формате .ply с последующей визуализацией внутри ноутбука. Таким образом, было показано, что Sonata не только запускается в среде Kaggle, но и позволяет получить наглядные результаты обработки облака точек.

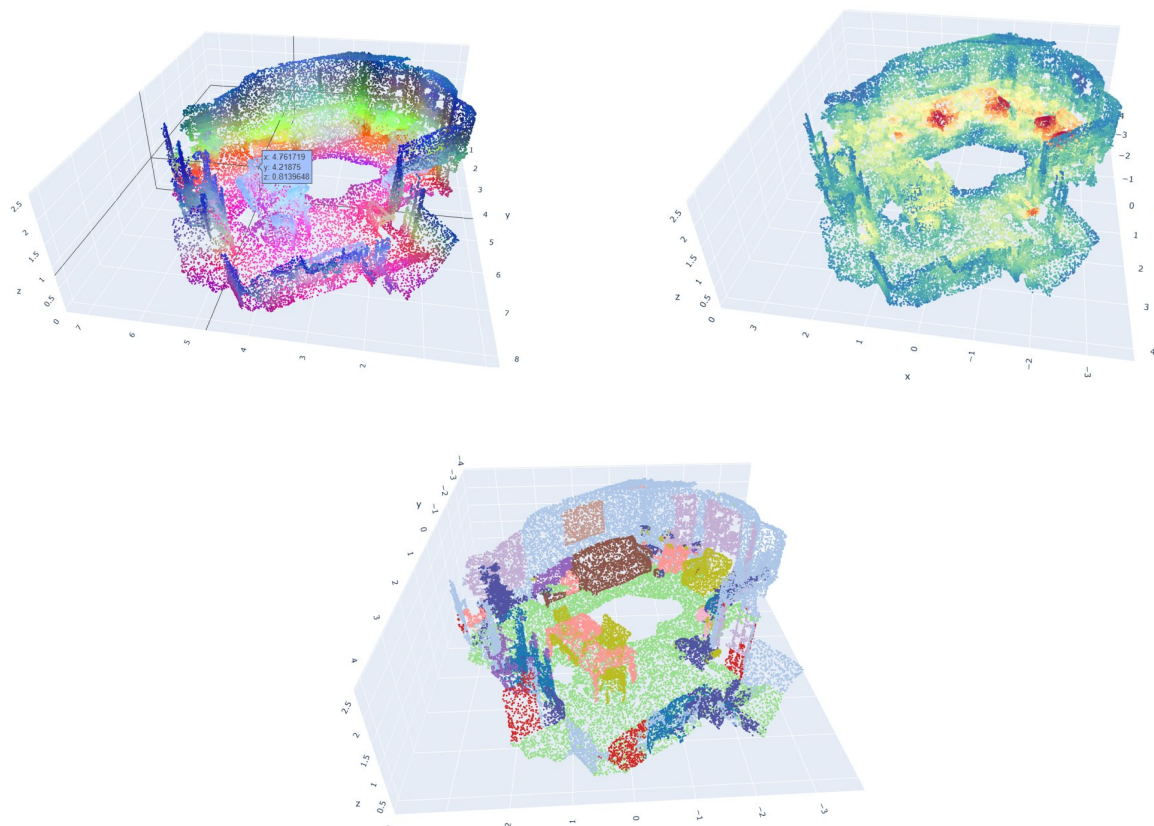


Рисунок 6.1: Визуализация Sonata

Помимо этого, была проведена первая попытка адаптации Sonata под задачу детекции и построения ограничивающих боксов вокруг предметов. Был реализован пайплайн, в котором после получения семантической сегментации точки одного класса объединяются в отдельные кластеры, а затем для каждого кластера строится bounding box. Однако на текущем этапе качество результатов не очень: в результате на облаке точек плохо различимы объекты, но боксы не выходят за границы выведенных облаков точек

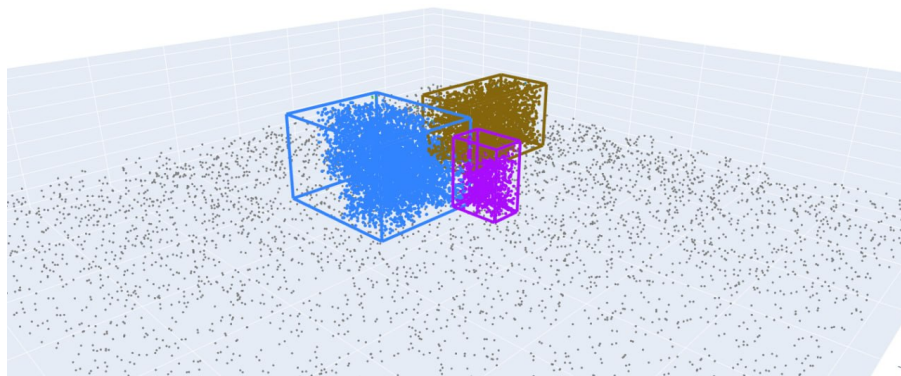


Рисунок 6.2: Sonata на детекцию

7 План дальнейшей работы

7.1 Демонстрационное приложение

Финальным результатом проекта планируется демонстрационное приложение для детекции объектов интерьера в реальном времени. Архитектура приложения включает три компонента:

Сбор данных. С помощью приложения Record3D на iPad или iPhone производится захват LiDAR-сканов помещения. Данные стримятся на компьютер по USB или локальной сети в формате кадровых облаков точек с цветом ($xyz + rgb$).

Сервер инференса. На сервере развёрнута обученная модель детекции, обернутая в API. Сервер принимает облако точек, запускает инференс и возвращает список обнаруженных объектов с координатами их 3D ограничивающих боксов, классами и уверенностью.

Визуализация. Полученные боксы визуализируются поверх облака точек. Планируется реализовать два варианта визуализации: интерактивный 3D-просмотр в браузере через Plotly или Three.js, и BEV-проекцию (вид сверху) с отмеченными объектами. В перспективе возможна визуализация прямо на iPad через AR-наложение боксов поверх реальной сцены средствами RealityKit.

7.2 Собственная тестовая выборка

Примерно через месяц планируется сбор собственной тестовой выборки. С помощью iPad LiDAR будут отсканированы 3–10 учебных аудиторий с разных углов и позиций. Для каждой сцены будет получено облако точек с цветом ($xyz + rgb$), после чего будет произведена разметка 3D ограничивающих боксов для объектов мебели: стулья, столы, доски, шкафы.

Собственная выборка позволит оценить качество модели на реальных indoor-сценах, характерных для целевого применения; выявить классы объектов, на которых модель работает хуже всего; и сформировать основу для дообучения в дальнейших итерациях. Разметка планируется в формате, совместимом с SUN RGB-D, что позволит совместно использовать оба датасета при обучении.

Список литературы

- [1] Яндекс Образование. Нейросети для облаков точек. 2026. URL: <https://education.yandex.ru/handbook/ml/article/nejroseti-dlya-oblakov-tochek> (дата обращения: 2026).
- [2] Qi C.R., Su H., Mo K., Guibas L.J. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation // arXiv preprint arXiv:1612.00593. 2016.
- [3] Qi C.R., Yi L., Su H., Guibas L.J. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space // arXiv preprint arXiv:1706.02413. 2017.
- [4] Zhao H., Jiang L., Jia J., Torr P., Koltun V. Point Transformer // arXiv preprint arXiv:2012.09164. 2020.
- [5] Wu X., Lao Y., Jiang L., Liu X., Zhao H. Point Transformer V2: Grouped Vector Attention and Partition-based Pooling // arXiv preprint arXiv:2210.05666. 2022.
- [6] Wu X., Jiang L., Wang P.-S., Liu Z., Liu X., Qiao Y., Ouyang W., He T., Zhao H. Point Transformer V3: Simpler, Faster, Stronger // arXiv preprint arXiv:2312.10035. 2023.
- [7] Wu X., DeTone D., Frost D., Shen T., Xie C., Yang N., Engel J., Newcombe R., Zhao H., Straub J. Sonata: Self-Supervised Learning of Reliable Point Representations // arXiv preprint arXiv:2503.16429. 2025.
- [8] Lang A.H., Vora S., Caesar H., Zhou L., Yang J., Beijbom O. PointPillars: Fast Encoders for Object Detection from Point Clouds // arXiv preprint arXiv:1812.05784. 2018.
- [9] gakkiri. simple-pointpillar: A simple PyTorch implementation of PointPillars. 2021. URL: <https://github.com/gakkiri/simple-pointpillar> (дата обращения: 2026).
- [10] Song S., Lichtenberg S., Xiao J. SUN RGB-D: A RGB-D Scene Understanding Benchmark Suite // CVPR. 2015.
- [11] Dai A., Chang A.X., Savva M., Halber M., Funkhouser T.A., Nießner M. ScanNet: Richly-annotated 3D Reconstructions of Indoor Scenes // CoRR. 2017.
- [12] Armeni I., Sener O., Zamir A.R., Jiang H., Brilakis I., Fischer M., Savarese S. 3D Semantic Parsing of Large-Scale Indoor Spaces // CVPR. 2016.
- [13] Geiger A., Lenz P., Urtasun R. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite // CVPR. 2012.
- [14] Yi L., Kim V.G., Ceylan D., Shen I.-C., Yan M., Su H., Lu C., Huang Q.-X., Sheffer A., Guibas L.J. A scalable active framework for region annotation in 3D shape collections // ACM Transactions on Graphics. 2016.
- [15] Caesar H., Bankiti V., Lang A.H., Vora S., Liong V.E., Xu Q., Krishnan A., Pan Y., Baldan G., Beijbom O. nuScenes: A multimodal dataset for autonomous driving // CoRR. 2019.
- [16] Abdelhalim G., Simon K., Bensch R., Parimi S., Qureshi B.A. LIDAR Warehouse Dataset. 2024. URL: <https://huggingface.co/datasets/Voxel51/lidar-warehouse-dataset> (дата обращения: 2026).
- [17] Yoshioka K. apple-lidar-stream: Stream Apple LiDAR data with Open3D. 2021. URL: <https://github.com/kentaroy47/apple-lidar-stream> (дата обращения: 2026).